💼 Web Developer Careers: Portfolio, Getting Hired and Freelancing

Fulllion Creative Works

Contents

1. What Employers Expect
2. Projects That Show Real Skill
3. Your Portfolio Site and GitHub Profile
4. Resumes and Applications
5. Interviews
6. Keeping Going Through a Long Search
7. Freelancing
8. Asking for a Raise
9. Staying Current
10. Common Mistakes
11. Sources

---

1. What Employers Expect

The minimum skills

These are the bare minimum to start job hunting, by path:

Path	Minimum to start applying
Front end	HTML, CSS and JavaScript, used confidently. Responsive layouts that work on phones. One framework, usually React. Git and GitHub.
Back end	One server language, such as Python, Java or JavaScript with Node.js. SQL and one relational database. One framework, such as Django, Flask or Express. Building a simple API. Git and GitHub.
Full stack	Enough of both to build, connect and deploy a small app on your own.

On top of that, almost every employer looks for:

- **Projects you can show,** live and on GitHub (Sections 2 and 3).
- **Problem solving:** breaking a task down and debugging with DevTools and error messages.
- **Communication:** explaining what you built and why, in writing and out loud.
- **Basics of accessibility, security and testing.** You don't need to be an expert, but you should know they matter.

> **Watch:** This is ALL you NEED to Become a Web Developer Today (2021) by **codeSTACKr** (December 2020) sets an even lower bar than the table: HTML, CSS and modern JavaScript, plus fetching data and reading JSON. He argues a framework helps but isn't required before you apply. For back end work, he suggests learning whatever language the employers near you use. His other points: give any tutorial project your own improvements before you show it, and apply even when you don't tick every requirement. Many junior ads now list a framework, so React is still worth learning, but don't wait for it. Read the video alongside the changes below.

What has changed since 2021

- **AI coding assistants are now normal.** In Stack Overflow's 2025 survey, 84% of respondents used AI tools or planned to. Yet 46% distrusted the accuracy of their output, compared with 33% who trusted it. Employers expect you to **read, check and explain** code, not just produce it, so solid fundamentals matter more, not less.
- **TypeScript is common.** It was used by 43.6% of all respondents in 2025, and 48.8% of professional developers. Learn it after JavaScript.
- **React is still the most used front end library** (44.7%), alongside Node.js (48.7%), and Next.js (20.8%) is now a common way to use React.
- **PostgreSQL is the most used database** (55.6%), so it's a safe SQL database to learn.
- **The fundamentals haven't changed:** HTML, CSS, JavaScript, SQL, Git and building real projects are still the core.

The outlook, in numbers

For the US, the Bureau of Labor Statistics reports:

- a **median pay of $99,520 a year** in 2025, for web developers and digital designers combined;
- **5% projected job growth** from 2025 to 2035, which it describes as faster than average;
- about **18% of web developers are self-employed,** which is why freelancing (Section 7) is a real option.

These are US national figures, so pay where you live, and for junior roles, can be quite different.

Expect a slower market than a few years ago. Indeed Hiring Lab's 2025 Q4 US Tech Labor Market Update found US software development job postings on Indeed were **31.8% below their February 2020 level** as of January 23, 2026, though roughly level with a year earlier (up 0.8%). Data and IT postings were down further. Fewer postings means more applicants for each one, which is why projects, tailoring and networking (Sections 2 to 6) matter so much.

The Odin Project's Getting Hired course

The Odin Project's free Getting Hired course is the best single walkthrough of the job search. It has two parts:

1. **Preparing for Your Job Search:** professional networking, strategy, what companies want, what you can do to prepare, and building your personal website.

2. **Applying to and Interviewing for Jobs:** collecting and qualifying job leads, building your resume, applying, interviewing, and handling a job offer.

Work through it alongside this guide.

---

2. Projects That Show Real Skill

A good portfolio project shows that you can build something real from start to finish, without a tutorial.

What makes a project stand out

1. **It's yours.** Built from a brief or your own idea, not copied from a tutorial. If it started as a tutorial, extend it with your own features.
2. **It's live.** A working link that a recruiter can click in ten seconds.
3. **It handles real data.** It calls an API, uses a database, or both.
4. **It works on phones** and can be used with a keyboard.
5. **It has a good README:** what it does, a screenshot, the tech used, how to run it, and what you'd improve next.
6. **It solves a problem someone has,** even a small one. Something for a club, a family business or your own daily routine makes a better story than another clone.

Fewer, bigger projects win. Internet Made Coder suggests two or three substantial projects that show a range of skills, rather than ten tiny ones. He also says to pick projects an interviewer can understand quickly (Section 4.2).

Where to find good project ideas

Source	What you get	Cost
Frontend Mentor	120+ challenges built from professional designs, across five difficulty levels. You build the design, post your solution, and get feedback from the community.	Free challenges. A paid Pro plan adds Figma design files, premium challenges and more AI code reviews.
roadmap.sh projects	Project ideas labeled beginner, intermediate or advanced. There are lists for frontend, backend and full stack. You can submit a solution and browse community solutions.	Free
The Odin Project	Projects built into the lessons, from beginner to full stack. See the examples below.	Free

Examples from The Odin Project, from easiest to hardest:

- **Foundations:** Recipes, Landing Page, Rock Paper Scissors, Etch-a-Sketch, Calculator.
- **JavaScript course:** Library, Tic Tac Toe, Restaurant Page, Todo List, Weather App, Battleship.
- **Node.js course:** Mini Message Board, Inventory Application, Members Only, File Uploader, Blog API, Messaging App, and Odin-Book (a small social network).

More front end project resources are in Learn Front End Web Development, Section 16, bigger capstone ideas are in Full Stack Web Development and Stacks, Section 10, and coding challenge sites are in Coding Practice and Interview Prep. Contributing to open source is another strong way to show real-world skill: see why contribute and your first contribution, step by step in Open Source and Developer Communities.

---

3. Your Portfolio Site and GitHub Profile

3.1 The portfolio site

Keep it simple and fast. A recruiter should understand who you are in a few seconds.

1. **A short intro:** your name, the kind of developer you are, and what you're looking for.
2. **Three to five of your best projects,** each with a screenshot, two or three sentences on what it does and what you used, a **Live** link and a **Code** link.
3. **A short "About" section,** including anything from a previous career that helps, such as customer service, design or teaching.
4. **Contact:** a contact form or a professional email address, plus links to GitHub and LinkedIn.
5. **Make the site itself a showcase.** It should be responsive, accessible and quick to load, because it's the first thing they'll judge.

For inspiration and honest feedback:

- JOB READY? JR. DEVELOPER PORTFOLIOS by **Joshua Fluke** (July 2019) reviews real junior portfolios. His main point: whoever opens your site gives it a few seconds, so every click counts.
  - Put your resume, contact details and each project's **Live** and **Code** links one click away. Don't make anyone hunt through your GitHub for the right repository.
  - Link to projects instead of embedding them in iframes, and give each repository a README.
  - Skip heavy background videos, which eat phone data and slow the page.
  - Test your site on very large screens as well as phones. One site he reviewed broke on a wide monitor.
  - A plain, fast page beats a clever one full of animation.
- Programmer Reacts to Inspiring Portfolio Websites 2024 by **Adrian Twarog** compares several real portfolios. Despite the title, it was published in June 2021. What he praises and criticizes:
  - **Show, don't tell.** Lead with what you do and who you help, not just your name in giant type.
  - **Say you're available,** for a job or freelance work, and give one clear call to action, such as **Contact me**.
  - **Make your email easy to find** and copy, not hidden behind a form.
  - **Write short case studies:** the problem, your process (sketches, wireframes), and the result. They beat a gallery of screenshots or logos.
  - **Avoid skill percentage bars.** Nobody knows what "60% back end" means. List the skills and point to work that uses them.

    - **Keep animation and text in check.** Motion that hides the links, walls of text, and low-contrast text all push visitors away. Never play sound automatically.

- The Odin Project's "Project: Building Your Personal Website" lesson, in Getting Hired.

> **Not only for developers.** The same advice works for 3D artists and game makers: lead with your best few pieces, keep the site fast, and make contact easy. Learn 3D Modeling ends its practice ladder with a portfolio piece, such as a lit scene or a turntable of your best model.

3.2 Putting it online

Option	Good for	Notes
GitHub Pages	A static portfolio of HTML, CSS and JavaScript	Name a repository `<username>.github.io` and your site appears at `https://<username>.github.io`. Sites can be up to 1 GB, with a soft limit of 100 GB of traffic a month. GitHub says Pages isn't for running an online business.
Firebase App Hosting, or another host	Sites with a back end, such as Next.js	See Connecting a Custom Domain to Firebase App Hosting for putting a site on your own domain.

A **custom domain** such as `yourname.dev` looks more professional, and you keep it if you change hosts. Domains cost a yearly fee from a registrar. GitHub Pages also supports custom domains.

3.3 The GitHub profile

Your GitHub profile is often the second thing an employer opens.

Add a profile README. This is a short introduction shown at the top of your profile:

1. On GitHub, click **+** (top right), then **New repository**.
2. Under **Repository name**, enter **exactly your GitHub username**.
3. Choose **Public**.
4. Turn on **Add README**, then click **Create repository**.
5. Click **Edit README**, write a few lines about yourself and what you're learning or building, and save.

GitHub only shows it if the repository is public, its name matches your username, and it has a non-empty `README.md` at the top level.

Pin your best work. On your profile, select **Customize your pins** in the **Popular repositories** or **Pinned** section, choose up to six repositories and gists, and select **Save pins**.

Tidy each pinned repository:

- A clear name and a one-line description.
- A README with a screenshot, a live link, and setup steps.
- A sensible commit history, with descriptive messages.

- **No secrets.** Never commit passwords, API keys or `.env` files. The custom-domain guide's Section 5.1 shows a `.gitignore` that keeps them out.

Publishing to GitHub with gh covers creating repositories and pushing code from the terminal.

---

4. Resumes and Applications

4.1 When are you ready to apply?

Probably sooner than you think. If you have the minimum skills in Section 1 and two or three solid projects online, start applying, and keep learning while you do. Coding Practice and Interview Prep has a readiness checklist too.

Joshua Fluke makes the case in When are you ready to apply to developer Jobs? (August 2019):

- **The moment you start asking "am I ready?", apply.** Nobody will tell you you're ready, and a rejection costs you nothing but time.
- **Requirements are a wish list.** Companies rarely find someone who meets all of them. The real filtering happens once you're talking to a person.
- **Treat an interview as a conversation in which you sell your work.** If you haven't used a tool at a job, show where you've used it in a project.
- **Never offer to work for less** just because you don't match every line of the ad.
- **Small startups** are often more flexible about hiring juniors. The startup job site he mentions, AngelList Talent, is now Wellfound.
- **The second half is about your first job, once you have it.** These are his opinions, so weigh them against your own situation:
  - If a new job turns out to be a bad place to work, you don't have to wait a year before looking elsewhere.
  - If you're just bored, first ask your manager for more responsibility or different work. Start looking if the answer is "that's your job".
  - When you resign, he says to turn down a counteroffer. His reasoning: your employer now knows you were looking, a raise doesn't fix why you wanted to leave, and you may be first in line when cuts come.

His salary figures are from 2019, so use current data (Section 8) instead.

No experience or CS degree? devsmak (now **James Cross**) gives five ways to find jobs that fewer people apply for, in Find WEB DEVELOPMENT JOBS with no experience or cs degree (June 2022):

1. **Use review sites, such as Glassdoor and Indeed, to find smaller local companies** that hire by word of mouth, and check small job boards. Then find the hiring manager and contact them directly.
2. **Note the other tech companies** in the same buildings and business parks, and research them too.
3. **Work with good local recruiters,** and ignore the mass-messaging ones. A real recruiter needs your resume and portfolio, not your Social Security number or bank details up front.

4. **Go to local meetups and conferences.** Speakers and recruiters there often mention openings.
5. **Use personal connections,** which he calls the best route. Tell friends and family you're looking. If your current employer needs a website or better emails, build a sample and offer to help part time.

He also suggests walking in at a company's front desk with a printed resume. Many offices now have controlled entry or staff working remotely, so a polite email or LinkedIn message is often the more practical first step.

> **Job scams are common.** The US Federal Trade Commission says honest employers never ask you to pay to get a job. It also warns about "employers" who collect your personal details or send a check and ask for part of it back. See the FTC's Job Scams page.

4.2 The resume

- **Keep it to one page,** with your projects near the top if you don't have developer experience yet.
- **Lead with results:** "Built a booking site used by a local gym" beats "Responsible for front end".
- **List skills you can back up** in an interview, not every tool you've touched.
- **Tailor it to each job.** Use the words from the job ad where they honestly apply, because many companies filter resumes automatically.
- **Use a simple layout.** Fancy columns and graphics can confuse those filters.

Two resume videos, and what they teach:

- This resume got me offers from Google, Microsoft, and Amazon! by **PIRATE KING** (September 2022) walks through his own resume with ten tips:
  - Keep it to one page.
  - Link your LinkedIn, GitHub and projects in the header.
  - Open with a skills section instead of a vague summary, and leave out obvious tools like Word.
  - Put experience above education.
  - Quantify results, for example "cut API response time from 10 to 2 seconds".
  - Explain the business purpose of what you built, because the first reader is usually a recruiter, not an engineer.
  - Keep old or unrelated jobs to one line.
  - Don't label yourself "beginner" next to a skill.

  He bases the one-page rule on the claim that recruiters spend about seven seconds on a resume. That figure comes from a 2018 eye-tracking study by Ladders (7.4 seconds on average), whose method was only thinly published, so treat it as a rough guide. He also says it's fine to "exaggerate" your impact. Be careful with that: every claim should be true and something you can explain in an interview. The video includes a paid bootcamp promotion.
- Why 97% of Programmer Resumes Get IGNORED by **Internet Made Coder** (September 2022) goes through a junior developer's resume line by line. The video doesn't back up the "97%" in its title. Its useful points:

    - A resume's only job is to get you an interview.
    - One page for juniors, ordered by relevance, describing what **you** did rather than what the team did.
    - In the US and UK, leave off your photo.
    - Start each bullet with an action verb, such as "built", "designed" or "led".
    - Show your skills inside your project bullets instead of a bare list of languages.
    - Tailor the resume to each job description, and keep the layout simple.

  He suggests estimating numbers when you don't have exact ones. Only use figures you can honestly defend. The video is sponsored by a web host.

Tools:

- **FlowCV** is a resume builder. Your first resume is free, with unlimited PDF downloads and 100+ templates, without watermarks. A paid Pro plan adds AI writing tools and more resume versions.
- Matty McTech's reel "An Easy way to Create your Résumé" (Facebook) shows the same kind of tool. You pick sections such as education, type your details, and watch the resume update in a live preview, then choose a template. FlowCV works the same way.
- The Odin Project's "Project: Building Your Resume" lesson in Getting Hired.

4.3 Cover letters and emails

Short and specific wins. Say which role you want, one or two things you've built that match it, and why that company. **Hemingway Editor** helps you cut long, hard-to-read sentences: it highlights them in yellow, and very hard ones in red. A paid Plus plan adds more.

4.4 Finding postings early

Applying soon after a job is posted can help, because fewer people have applied yet.

The "early applicant" LinkedIn trick. Matty McTech's Facebook reel "Try this Job Search trick out to be one of the first to apply!" says to type this into LinkedIn's job search, with your own job title:

```
"early applicant" AND Software Developer
```

The reel promises you'll be among the first to apply. **What it really does:**

- **It changes the results.** In a signed-out US search in November 2026, all 60 visible results carried LinkedIn's **Be an early applicant** label. A plain "Software Developer" search showed the label on only 10 of 60.
- **But the jobs weren't necessarily new.** Many had been posted two weeks to a month earlier, and most were senior roles, so "early applicant" isn't the same as "just posted".
- **LinkedIn doesn't document it.** Its help pages explain that quotes match an exact phrase and that `AND` must be in capitals, but they don't explain what earns the label or say the phrase works as a filter. Signed-in results, other countries and later changes to LinkedIn may give different results.

So it's a harmless extra search to try, not a guarantee. The filters LinkedIn does document are more dependable:

1. Search for your job title and location.
2. Under **Date posted**, choose **Past 24 hours** (or past week).
3. Turn on the **Under 10 applicants** filter, which LinkedIn says shows jobs that currently have fewer than 10 applicants. It may sit under **All filters**.
4. Turn on the **job alert** toggle on the results page, so new matches come to you. Then select **Manage alerts** to choose how often, and whether by email or LinkedIn notification. On mobile, alerts default to daily.

Don't let speed hurt quality. A tailored application a day later beats a rushed one.

4.5 Signing documents

Offer letters and freelance contracts often need a signature. PiXimperfect shows how to make a clean digital version of your handwritten signature in Make Your Signature Digital with Photoshop!. In short:

1. Sign a few times on white paper, then photograph or scan the best one.
2. In Photoshop, crop it, and open the **Channels** panel. Pick the red channel, which usually has the most contrast.
3. Use **Curves** to make the paper pure white and the ink solid.
4. Load that channel as a selection, invert it so only the ink is selected, and fill it with a **Solid Color** layer.
5. Hide the original layer, and save as a PNG with a transparent background.

Photoshop is paid, and many PDF readers have their own fill-and-sign tools: see PDF and Document Tools. Keep your signature image private, and don't post it anywhere public.

---

5. Interviews

Web developer interviews usually include some mix of:

1. **A screening call:** your background, and why this job.
2. **A technical round:** live coding, a take-home project, or talking through code you've written.
3. **A behavioral round:** how you work with others, handle mistakes and learn.
4. **Questions for them:** always have a few ready.

Two things that help with every round: **know your own projects inside out,** because you'll be asked to explain your choices, and **think out loud** when solving problems. Joshua Fluke's point (Section 4.1) is that a coding problem tests how you communicate under pressure as much as the answer. If you're stuck, say where, and what you'd try next, and ask for a hint. ThePrimeagen (Section 6) suggests practicing by interviewing at companies you care less about, so the ones you want aren't your first try.

Interview preparation now has its own guide: Coding Practice and Interview Prep covers practice sites, LeetCode and NeetCode, *Cracking the Coding Interview*, solving a problem out loud, and behavioral interviews with the STAR method. The Odin Project's Getting Hired course also has lessons on preparing to interview and handling a job offer.

---

6. Keeping Going Through a Long Search

A first developer job can take months of applications. That's normal, and it isn't a judgment of your worth.

- **Set realistic expectations.** In How am I suppose to get a job as a software engineer? (June 2022), ThePrimeagen admits that long requirement lists and algorithm interviews aren't fair, then argues you should treat them as a checklist:
  - Most listed technologies are a wish list, not hard requirements.
  - If algorithms and data structures are the gate, put in a few weeks of study. They get easier with practice.
  - Learn enough of each listed tool, such as Docker, to discuss how it's used and its trade-offs. Employers mostly want someone who can find and build the solution, not an expert in everything.
  - Go to meetups, talk ideas through with other developers, and practice interviewing.
- **Rejection is part of the process.** In The pain of not being able to find a software job (March 2023), KC tells his own story:
  - He assumed his computer science degree would be enough, and didn't prepare for interviews.
  - He sent around 600 applications over several months, and got very few replies.
  - He finally landed a job at a five-person startup.
  - His lessons: applying is partly a numbers game and partly luck, so prepare before interviews rather than after, and make small progress every day.
- **Treat it like a project.** Track your applications in a spreadsheet, aim for a steady number each week, and review what's working.
- **Keep building.** A new project or open source contribution gives you something fresh to talk about.
- **Look after yourself.** Sleep, breaks and exercise keep you going for the months a search can take. Learning How to Learn Programming lists the signs of burnout to watch for.
- **Network.** Tell people you're looking. Many jobs are filled through referrals.

When the market is slow, Codewars' blog post How to get hired as a software developer during a tech recession (Ellen Weaver, February 2023) suggests:

1. Take an honest inventory of your strengths and weaknesses.
2. Learn the skills that fill your gaps, including communication.
3. Network actively, and tell your contacts how they can help.
4. Look beyond big tech, at industries such as finance, agriculture and manufacturing.
5. Consider freelancing for income and real experience (Section 7).

6. Keep your resume, portfolio, LinkedIn and GitHub up to date.

Related roles can be a way in, too. Help desk and IT support jobs teach troubleshooting and working with people, and some developers start there. IT Support Careers and Certifications covers that path, which can lead on to cloud and DevOps work.

---

7. Freelancing

Freelancing means working for clients yourself, project by project. It can be a whole career, a side income, or a way to build experience while you job hunt.

7.1 Two videos to start with

How to Start Freelancing | with HTML + CSS by **Nicole Young** (January 2021; the channel is now **A Good Living with Nicole Young**). She started freelancing in early 2020 knowing only HTML and CSS, and argues you don't need to wait until you're advanced. Services she suggests at that level:

- **Custom HTML email templates** for businesses whose email marketing needs more than a drag-and-drop editor.
- **Website upkeep:** making day-to-day content and style updates to an existing site.
- **Customizing WordPress or Squarespace sites** beyond what their built-in settings allow. That's the path she took.
- **Combining skills,** such as web updates plus marketing or social media, for solo business owners.

Her other point: client work shows you which skills are in demand and pushes you to learn faster. Stay within what you can deliver.

Ultimate Freelancing Guide for Web Developers on **freeCodeCamp.org** (January 2020, about three hours) is taught by freelance web developer **Luke Ciciliano**. Attorney **Cari Ross** (Ross Legal Services) covers contracts, and bookkeeper **Erin Lehr** (KPI Bookkeeping) covers money. It's US-based and aimed at running freelancing as a real business. Jump to:

Part	Start
Three rules for making money	3:20
Picking a niche	21:32
Choosing your services and pricing	28:56
Contract basics (Cari Ross)	1:00:10
Marketing startup tasks	1:12:15
Finding clients, short term	1:34:39
Building the business, long term	1:47:41
Accounting and money (Erin Lehr)	2:12:14
Working efficiently	2:44:03

Its core advice:

- **Sell the result, not the tech.** Clients pay for the problem you solve. A hand-coded site isn't worth more to them than a well-made CMS site unless you can show why.
- **Pick a niche and say no to work outside it.** Reusing code and industry knowledge makes each later job faster, so a flat fee earns you more per hour.
- **Charge flat fees, and add recurring income** such as hosting and maintenance.
- **Treat it as a full-time job.** When client work doesn't fill the week, spend the rest finding the next client.
- **First clients:**
  - Tell friends and neighbors once, without pestering.
  - Build complete profiles with work samples on marketplaces such as Upwork, Fiverr and Thumbtack, and expect fees and low prices there.
  - Say you're open to freelance work on LinkedIn.
  - Network with people who serve the same clients you want, such as marketers, rather than with other developers.
- **Long term:** Meet deadlines, and warn clients about a delay before it happens. Referrals and repeat work follow. A blog or how-to videos that answer your clients' questions bring in leads for years.
- **Money:** Keep a separate business bank account, and set aside 25 to 30% of what comes in for tax (the bookkeeper's rule of thumb, not a tax rate). Do your bookkeeping weekly.

What has changed since 2020:

- **Renamed products:** G Suite is now Google Workspace, and Office 365 is now Microsoft 365.
- **Wave:** The accounting app was free then. It now has a free Starter plan and a paid Pro plan.
- **LinkedIn skill tests:** LinkedIn has retired the skill assessments he suggests taking.
- **1099 forms:** The $600 threshold mentioned in the accounting part rose to $2,000 for US payments made from 2026 (Section 7.5).
- **LLCs:** The video suggests an LLC saves self-employment tax. In the US, a single-member LLC is taxed like a sole proprietor by default. Any saving comes from electing a different tax treatment, which an accountant should advise on.
- **Hustle:** The long-hours advice and the confident income promises near the end are one person's view.

7.2 Finding clients

1. **Start with people you know.** Friends, family, former employers, clubs and local businesses often need a site or a fix. Your first few clients usually come from your network.
2. **Look for sites that need help.** Local businesses with slow, broken or phone-unfriendly sites are good candidates. Contact them politely, with a specific idea, not a generic pitch.
3. **Ask every happy client for a referral** and a short testimonial for your portfolio.

4. **Freelance marketplaces** such as Upwork can bring work, but they take a fee from your earnings, competition is strong, and some charge you to send proposals. Check each platform's current fees before you sign up.
5. **Specialize when you can.** "Websites for local restaurants" is easier to market, and to charge more for, than "I build websites".

> **Watch for scams.** Be wary of anyone who wants you to pay to get work, sends a check and asks for part of it back, or pushes you to move payment off a platform you met on.

7.3 Pricing

There's no single right price, but these points help you avoid charging too little:

- **Hourly or per project.** Hourly is simpler when the scope is unclear. A fixed project price rewards you for working efficiently, but only if the scope is written down.
- **Your rate has to cover more than your coding time.** It also pays for emails, meetings, quotes, invoicing and learning, plus your tools, equipment, any health cover, time off, and the taxes an employer would otherwise handle.
- **Find out what others charge** for similar work in your area or niche, and use that as a range.
- **Take a deposit up front,** and split larger projects into milestones you're paid for as you go.
- **Charge for changes outside the agreed scope.** Write down what's included, and quote extra for anything else.
- **Offer ongoing care,** such as hosting, updates and small fixes, as a monthly fee. It brings steadier income.

7.4 Contracts

Always have a written agreement, even with friends. It protects both of you. A basic web project contract usually covers:

Item	What to write down
Scope	Exactly what you'll build, how many pages and features, and what's not included
Deliverables and timeline	What the client gets, and when, including what you need from them (text, images, logins)
Revisions	How many rounds of changes are included
Payment	Price, deposit, milestones, due dates, and what happens if payment is late
Ownership	Who owns the code and design, and when it passes to the client, often on final payment
Hosting and accounts	Whose name the domain and hosting are in. It's usually best that the client owns them.
Ending early	How either side can end the project, and what's paid for work done so far

Templates are a good starting point, but rules on contracts differ between countries and states. For big projects, have a local lawyer check yours.

7.5 Taxes and registering

When you freelance, nobody withholds tax from your pay, so **set money aside from every payment.** Keep records of your income and business expenses from day one. Some general examples:

- **United States:** You're automatically a sole proprietor when you do business on your own, according to the SBA. The IRS says you must pay self-employment tax (Social Security and Medicare, 15.3% in total) if your net earnings from self-employment are $400 or more, on top of income tax. You generally pay estimated tax each quarter with Form 1040-ES, and report your business on Schedule C and Schedule SE. A business client that pays you $2,000 or more in 2026 generally sends you a Form 1099-NEC (the threshold was $600 for payments before 2026). You still report all your business income, with or without that form.
- **United Kingdom:** GOV.UK says you must register for Self Assessment as a sole trader if you earn more than £1,000 from self-employment in a tax year (6 April to 5 April).
- **Everywhere else:** Check your national tax authority's site for self-employment rules, and whether you need to register a business or charge sales tax or VAT.

> **This is general information, not tax advice.** Rules change and depend on your situation. An accountant can save you more than they cost, especially in your first year.

---

8. Asking for a Raise

Once you're in a job, your pay won't always rise on its own. Three short videos share one idea: turn the request into a conversation with your manager, not a one-time demand.

- **Jefferson Fisher**, "Best way to ask for a pay raise." (Facebook). The tip: ask your manager what they would do if they wanted to earn more there, or how they reached their own role. Most people enjoy giving their view, and the answer gives you a roadmap.
- **Jefferson Fisher** (@justaskjefferson), "This is how to ask for a raise without feeling awkward." on TikTok. Two tips:
  - Ask first whether you can talk about your salary without it hurting your working relationship, which separates the money from the relationship.
  - Say it's a conversation you'd like to have over the next few weeks. That way it doesn't sound like an ultimatum or a threat to leave, and your manager has time to work on it with you.
- **Nick Ayala** (@nickjayala), "Ask This Question Instead of Asking for a Raise" on TikTok. The clip has music but no speech, and the advice is in its on-screen text and caption. His idea: name the pay you want to reach by a set date, then ask, "What would have to be true for that to happen?" Your manager then defines the goals, and people tend to back what they helped build.

These match mainstream advice. Recruiting firm Robert Half's How to Ask for a Raise also suggests asking, if the answer is no, what milestones would justify an increase, and agreeing on a timeline to review it. The steps below draw on it:

1. **Collect evidence of your impact.** Keep a running list of what you've shipped, problems you've solved, and any numbers, such as faster pages, fewer bugs or time saved. Note new responsibilities you've taken on since your last raise.
2. **Research market pay** for your role and location. In the US, the BLS Occupational Outlook Handbook gives national figures, and salary surveys and job ads give local ranges.
3. **Put it on one page.** A short written summary of your results and the market data is easy for your manager to pass on for approval.
4. **Time it well.** Ask ahead of your performance review or budget planning, or soon after a clear win, not during a crisis.
5. **Practice, then ask for a meeting.** Have a specific number or range in mind, and state it plainly, without hedging.
6. **If the answer is "not now",** ask what would need to happen, and by when, then agree on a date to review it. If the budget is the problem, you can ask about alternatives such as a one-time bonus or paid training.
7. **Get it in writing.** Follow up by email to confirm what was agreed: the amount, the goals, and the date.

---

9. Staying Current

You don't need to chase every new tool. An hour or two a week is plenty.

- **MDN Blog** for new web platform features, explained well.
- **Baseline on web.dev** tells you which features are safe to use. "Newly available" means they work in all the core browsers, and "widely available" means 30 months have passed since then. It's defined by the WebDX Community Group.
- **State of JS** and the **Stack Overflow Developer Survey**: yearly surveys showing which tools developers use and like.
- **roadmap.sh** roadmaps are updated by the community as the field changes.
- **Release notes and blogs** for the tools you use, such as React, Next.js and Node.js, and the GitHub Changelog.
- **Communities** of other developers: see Open Source and Developer Communities.

Learn new things by building with them in a small project, the same way you learned at the start.

---

10. Common Mistakes

Mistake	Why it hurts	Do this instead
Waiting until you "know enough" to apply	There's always more to learn, so the moment never comes	Apply once you meet Section 1's minimum, and keep learning

Mistake	Why it hurts	Do this instead
A portfolio full of tutorial clones	Recruiters see the same to-do apps every day	Extend them, or build from a brief (Section 2)
Projects without live links or READMEs	Most people won't clone and run your code	Deploy everything, and write a README with a screenshot
Secrets committed to GitHub	Bots scan public repositories for keys	Use `.gitignore`, and rotate any key you've exposed
The same resume for every job	It doesn't match the ad, so it gets filtered out	Tailor it each time (Section 4.2)
Inflated numbers on your resume	Interviewers ask you to explain them	Use only figures you can honestly back up (Section 4.2)
Freelancing without a contract	Scope grows and payment slips	Write down scope, price and payment terms first (Section 7.4)
Spending all of every freelance payment	The tax bill arrives later	Set aside a share of every payment (Section 7.5)

---

11. Sources

Videos and posts credited

- codeSTACKr, This is ALL you NEED to Become a Web Developer Today (2021) (December 2020)
- Adrian Twarog, Programmer Reacts to Inspiring Portfolio Websites 2024 (June 2021)
- Joshua Fluke, JOB READY? JR. DEVELOPER PORTFOLIOS (July 2019) and When are you ready to apply to developer Jobs? (August 2019)
- devsmak (James Cross), Find WEB DEVELOPMENT JOBS with no experience or cs degree (June 2022)
- PIRATE KING, This resume got me offers from Google, Microsoft, and Amazon! (September 2022)
- Internet Made Coder, Why 97% of Programmer Resumes Get IGNORED (September 2022)
- PiXimperfect, Make Your Signature Digital with Photoshop!
- ThePrimeagen, How am I suppose to get a job as a software engineer? (June 2022)
- Created By KC, The pain of not being able to find a software job (March 2023)
- A Good Living with Nicole Young, How to Start Freelancing | with HTML + CSS (January 2021)
- freeCodeCamp.org, Ultimate Freelancing Guide for Web Developers, by Luke Ciciliano with Cari Ross and Erin Lehr (January 2020)
- Matty McTech, "An Easy way to Create your Résumé" (Facebook) and "Try this Job Search trick out to be one of the first to apply!" (Facebook)
- Jefferson Fisher, "Best way to ask for a pay raise." (Facebook) and "This is how to ask for a raise without feeling awkward." (TikTok)
- Nick Ayala, "Ask This Question Instead of Asking for a Raise" (TikTok)
- Codewars blog, How to get hired as a software developer during a tech recession

Learning and projects

- The Odin Project, Getting Hired, Foundations, JavaScript and NodeJS courses
- Frontend Mentor
- roadmap.sh, Project ideas

GitHub

- Managing your profile README
- Pinning items to your profile
- What is GitHub Pages?, GitHub Pages limits and Quickstart

Job search tools

- FlowCV, Hemingway Editor and its readability help
- LinkedIn Help, Use Boolean search on LinkedIn, Filter and sort job search results, Job alerts on LinkedIn and Skill Assessments: no longer available
- LinkedIn job search for `"early applicant" AND Software Developer` and for `Software Developer`, signed out, United States, November 2026
- Wellfound, AngelList Talent is now Wellfound
- US Federal Trade Commission, Job Scams
- HR Dive, Eye tracking study shows recruiters look at resumes for 7 seconds (November 2018, on the Ladders study)
- Robert Half, How to Ask for a Raise

Figures, tax and business

- Stack Overflow, 2025 Developer Survey and Technology
- US Bureau of Labor Statistics, Web Developers and Digital Designers
- Indeed Hiring Lab (Daniel Culbertson), 2025 Q4 US Tech Labor Market Update (PDF, data through January 23, 2026)
- IRS, Self-Employed Individuals Tax Center, Self-Employment Tax, Form 1099-NEC and independent contractors, Manage taxes for your gig work and Limited Liability Company (LLC)
- Wave, Pricing
- SBA, Choose a business structure
- GOV.UK, Set up as a sole trader and Working for yourself

Staying current

- MDN Blog, Baseline, State of JS, roadmap.sh, GitHub Changelog

---

Latest version: github.com/ImmuneMoon/Web-Developer-Careers-Portfolio-and-Freelancing

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.